Politecnico di Torino

2025/26

Deterministic Timeline-Based Scheduler for FreeRTOS

Operating Systems for Embedded Systems

Bruno Samuel	S363583
Carbone Gaetano	S359776
Deplano Lorenzo	S361511
Ionita Elsa Francesca	S361804

Group 47

Project 1

OVERVIEW

The project implements a timeline-based scheduler that replaces the FreeRTOS priority scheduler.

Scheduling is organized in **Major Frames** subdivided into **Sub-frames** with different type of tasks. **Hard Real-Time tasks,** assigned at fixed slots and **Soft Real-Time tasks** that run in idle time in a fixed order and are preemptible by HRT.

Key Specifications

At each Major Frame end all tasks are reset to guarantee deterministic, repeatable execution.

A lightweight trace records task start/end ticks, deadline misses and CPU idle time for verification.

Non-Functional Requirements

Jitter ≤ 1 tick
Overhead ≤ 10% CPU (≤8 tasks)
Coding guidelines: FreeRTOS coding guidelines.

CONTENTS

- Deterministic Timeline-Based Scheduler
- Temporal Structure
- Task Categories
- Test-Driven Development

Standard FreeRTOS Scheduler

Event-Driven: Reactive to priorities
Dynamic logic, calculated at runtime
High **jitter** (variable task start times)
The **Idle Time** is used by any lower priority task

```
PRIORITY  |
          |       [T_High]          [T_High]          [T_High]
   HIGH   |-------|      |----------|      |----------|      |----
          |       |      |          |      |          |      |
          | [T_Low]      |    [T_Low]      |    [T_Low]      |
   LOW    |--|    |------|----|     |------|----|     |------|----
          |  |    |      |    |     |      |    |     |      |
   TIME   |--|----|------|----|-----|------|----|-----|------|--->
             ↑    ↑
         Event   Preemption
```

Deterministic Timeline-Based Scheduler

Time-Triggered: Proactive via schedule
Static logic, predefined at compile time
Zero/Low **jitter** (fixed execution slots)
The **Idle Time** is used specifically for Soft Real Time

```
                |---- MAJOR FRAME (Cycle 1) ----|---- MAJOR FRAME (Cycle 2) ----|
                |--SF1--|--SF2--|--SF3--|--SF4--|--SF1--|--SF2--|--SF3--|--SF4--|
                |       |       |       |       |       |       |       |       |
Tasks (HRT): [TaskA]  IDLE    IDLE   [TaskB] [TaskA]  IDLE    IDLE   [TaskB]
Tasks (SRT):         (TaskX) (TaskY)                 (TaskX) (TaskY)
```

Temporal Structure

Major Frame: The global cycle that repeats indefinitely.
Sub-frames: Smaller slots of the Major Frame used to group tasks and manage task execution windows.

Tasks are mapped to specific Sub-frames. The scheduler executes this exact sequence cyclically.

```
| -- SF1 --|-- SF 2 --|-- SF 3 -- | -- SF1 --|-- SF 2 --|-- SF 3 --|
|--------MajorFrame --------- | -------MajorFrame ---------|
```

Temporal Structure

Implementation

```
typedef struct {
    uint32_t ulMajorFrameTicks;
    uint32_t ulNumSubframes;
} TimekeeperConfig_t;

typedef struct {
    uint32_t ulGlobalTick;
    uint32_t ulMajorFrameTick;
    uint32_t ulCurrentSubframe;
    bool xMajorFrameRestart;
} TimekeeperState_t;
```

Temporal Structure

Implementation

```
typedef struct{
    const char* pcName;
    TaskFunction_t pxTaskCode;
    void* pvParameters;
    uint16_t usStackDepth;
    TimelineTaskType_t eType;
    uint32_t ulStartTick;
    uint32_t ulEndTick;
    uint32_t ulSubFrameId;
} TimelineTaskConfig_t
```

Static Scheduling Table:
All timing parameters (offsets, durations, ...) are stored in a TimelineTaskConfig_t data structure.

Temporal Structure

Implementation

```
typedef struct{
    BaseType_t xActive;
    uint32_t ulMajorFrameTicks;
    uint32_t ulCurrentFrameTick;

    // --- HRT Management --- //
    HRTEntry_t *pxHrtTable;
    uint32_t ulHrtCount;

    // --- SRT Management --- //
    SRTEntry_t *pxSrtTable;
    uint32_t ulSrtCount;
    volatile uint32_t ulCurrentSrtIndex;
    BaseType_t xCurrentSrtCompleted;
} TimelineControlBlock_t;
```

Static Scheduling Table:
All timing parameters (offsets, durations, ...) are stored in a TimelineTaskConfig_t data structure.

System Initialization:
vConfigureScheduler() function parses the table, initializes hardware timers, and prepares the FreeRTOS environment.

Task Categories

Hard Real-Time

Strict Start/End time enforcement.

The scheduler knows the start time and end times of each task within the sub-frame.

- **Non-preemptive** during its slot to ensure atomic execution.
- **Run until completion** or **killed if it exceeds** its allocated deadline.

```
typedef struct{
    char* pcTaskName;
    TaskHandle_t xTaskHandle;
    uint32_t ulStartTick;
    uint32_t ulEndTick;
    uint32_t ulDuration;
    HRTStatus_t eStatus;
    uint32_t ulSubFrameId;
} HRTEntry_t;
```

Task Categories

Soft Real-Time

Executes only **during Idle Time left by HRT tasks**. They are scheduled in a fixed compile-time order.

Preemptible by any incoming HRT task to maintain timeline integrity and there's **no guarantee of completation** within the frame.

```
typedef struct{
    char* pcTaskName;
    TaskHandle_t xTaskHandle;
    SRTStatus_t eStatus;
} SRTEntry_t;
```

INTEGRITY

FreeRTOSConfig.h

```
#define configUSE_TIMELINE_SCHEDULER 1
```

Safe update the FreeRTOS kernel
modified the functions **vTaskSwitchContext**, **xTaskIncrementTick** to manage various types of tasks and handle the tick.

tasks.c

```
#if (configUSE_TIMELINE_SCHEDULER == 1)
...
#endif
```

Automated Test Framework

Automated Pipeline
A script manages dynamic compilation, execution, and output verification.

Strict Temporal Validation
Python scripts parse **execution logs** to ensure task state changes occur at the exact **expected system tick**.

Validated Scheduling Scenarios

Nominal Behavior

Preemption & Resumption

Fault Tolerance

Nominal Behavior

HRT Precision:
Hard Real-Time tasks start at exact pre-configured ticks and yield the CPU immediately upon completion.

SRT Background:
Soft Real-Time tasks seamlessly consume available idle gaps.

Preemption & Resumption

Strict Priority:
HRT time-slots instantly preempt any running SRT task.

Context Recovery:
Preempted SRT tasks accurately resume execution in the next available timeline gap.

Fault Tolerance

HRT tasks caught in infinite loops are forcefully aborted exactly at their deadline boundary.

An aborted task does not crash the system; the next scheduled task starts perfectly on time.

Politecnico di Torino

2025/26

DEMO

Group 47

Project 1

Project division

1. **Configuration and data structures**
2. **Timekeeper**
3. **Hard Real-Time Scheduling**
4. **Deadline Management and Kill Policy**
5. **Soft Realt-Time Scheduling**